Optimized Simulation of Discrete Rank based Scheduling

Amishi Gupta, Jasmine Si, Michael Sidoti

Cornell ORIE 4999, Spring 2026

Contents

1 Introduction **2**

2 Background/System model **2**

3 Simulating Discrete Rank functions **3**
3.1 Naive approach: discrete event simulation . . . 3
3.1.1 Pitfall of discrete event simulation . . . 4
3.2 Our design: bisection-driven discrete event simulation . . . 5
3.2.1 The Core Algorithm Overview . . . 5
3.2.2 Group Advance . . . 5
3.2.3 Bisection . . . 6
3.2.4 Putting it all together . . . 7

4 Results **7**
4.1 Comparing Performance on Varying Resolution Discrete Rank Data . . . 8
4.2 Comparing Performance on Event driven LAS . . . 9
4.3 Comparing Performance on Varying Number of Jobs . . . 9

5 Conclusion **11**

Acknowledgements **11**

References **11**

1 Introduction

A queue can be thought of as any system that has to process a number of entities that are waiting to use some resource. For our purpose, we will refer to the resource as a server and the entities waiting to use it as jobs. Furthermore, we will be investigating a single-server queue, where only one job can be served at any moment as opposed to a multi-server queue, where multiple servers can serve many different jobs at one time. An intuitive followup to this description is wondering about how the queue decides which job to serve next. The answer to this is based upon the queue's scheduling policy. For example, many queues in real life, like the line to checkout of a store, follow a FIFO scheduling policy, where the earliest person who arrives in line is the next one to checkout. One metric we can look at to analyze the efficacy of a scheduling policy is the mean delay. The mean delay is simply the average time that a job stays in the queue before completing. A low mean delay of a scheduling system means that each job did not have to wait very long in the queue which is almost universally seen as a good thing. Throughout the course of this project we investigated different scheduling policies. Towards the end, our group was mainly focused on optimizing the algorithm for queues that functioned under a discrete-rank function, a scheduling policy that prioritizes jobs based on how long the server has served them.

2 Background/System model

To begin our project, we first defined our basic queuing model; each job j has an arrival time a_j, and a size s_j. The server will run one job at a time, with rate = 1. For example, if the server runs a job j with size $s_j = 5$ for 5 units of time, the job will be completed. For each job j we can define their attained service x_j as how much time the server has run job j at time t. If a job j has size $s_j = 8$ and $x_j = 4$ at a fixed time, the remaining work to complete job j, $r_j = s_j - x_j = 4$. For the rest of this project we will be referring to a jobs attained service as its age.

A rank function is a function $r(x) : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$ whose input is an age, and output is a rank. For a list of jobs, we can input all of their ages into r, and assign each job a rank r. In order for the scheduler to decide which job j the server will run, it will always choose the job with the lowest rank.

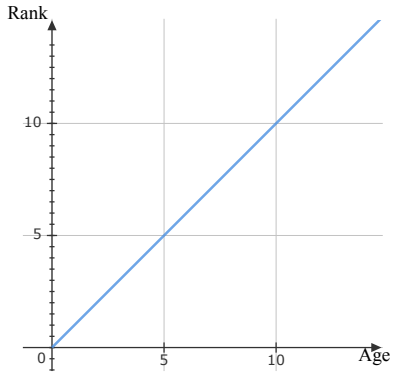


Figure 2.1. Rank function for LAS

For example, a possible rank function for a very popular scheduling model, Least Attained

Service (LAS), is $r(x) = x$. The rule of LAS is to simply serve jobs with the least age so far. Notice that this function is increasing monotonically. This means for greater values of age, we always get a greater rank. Thus, a simulator using this policy will always serve the jobs with the least age, as those are the jobs with the lowest rank.

Our group mostly studied discrete rank functions, a type of rank function where at a discrete set of ages, there's a nonzero rank, and it has rank 0 in between. These rank functions are easily representable in code, especially when compared to continuous rank functions. We can represent our rank function list of tuples $[(a_1, r_1), (a_2, r_2), ..., (a_n, r_n)]$ such that $r(x) = \begin{cases} r_i & \text{if } x = a_i \\ 0 & \text{otherwise} \end{cases}$ where $1 \leq i \leq n$.

See figure 2.2 for discretized LAS, with increments of 0.5. Here, we have the rank data as $[(0, 0), (0.5, 0.5), (1, 1), ..., (5, 5)]$.

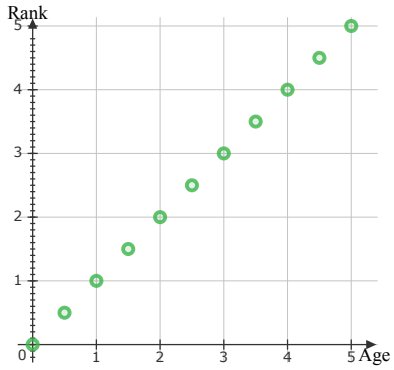


Figure 2.2. Discrete Rank function for LAS

3 Simulating Discrete Rank functions

In order to talk about simulating discrete rank functions, we will first need to define what our simulator does. Our simulator took in a discrete rank function and a list of jobs that needed to be served. The simulator is a while loop that only terminates when there is no job left to be served. It would keep track of a global time T and the time it takes for each job to complete starting from when it arrived. We will call this time the job's delay. For example, if a job arrives at $T = 2$ and completes at $T = 6$, its delay will be $6 - 2 = 4$. The output is a list of response times, one for each job.

3.1 Naive approach: discrete event simulation

Our first instinct in simulating discrete rank functions was to handle each important moment in the simulation as a separate event. We decided that the most important events would be:

1. When the job being served changes ranks. (We will call this dt_{change}).
2. When the job being served completes. (We will call this $dt_{complete}$).
3. When we reach a time t where there is an arrival. (We will call this $dt_{arrival}$).

From this, the structure of our simulator naturally follows. We identify the currently running job, the one with the lowest rank, and compute the time it would take to reach each of these three

important events. We can then take the minimum of times to figure out which event would happen next and advance our global simulator time by $\min(dt_{change}, dt_{complete}, dt_{arrival})$.

Next, our group needed to figure out what we would do in each of these cases. In each of these cases, the simulator will serve the job it is running by dt units, add dt to our global time T, and go back to the top of our while loop; however in the event of a completion and arrival, some additional logic needs to be done. If the next event is a completion, our simulator will keep track of job j's completion which is at time $T + dt_{complete}$. If the next event is an arrival, the simulator will add the job that is arriving to a list of jobs in the system before returning to the top of the while loop. This algorithm can be represented in the pseudocode below:

Algorithm 3.1. Naive Event-Driven Simulation

```
1:  while unfinished jobs remain do
2:      if S = ∅ then
3:          advance time to next arrival                    ▷ server idle
4:          add arriving jobs to S
5:          continue
6:      end if
7:      j* ← arg min_{j∈S} rank(j.age)                      ▷ lowest-rank job runs
8:      dt_bp ← time until next breakpoint of j*            ▷ rank changes
9:      dt_complete ← j*.remaining                          ▷ job completes
10:     dt_arrival ← time until next arrival                ▷ new job arrival
11:     dt ← min(dt_bp, dt_complete, dt_arrival)            ▷ next event
12:     advance simulation time by dt
13:     j*.age ← j*.age + dt
14:     j*.remaining ← j*.remaining − dt
15:     if dt = dt_complete then
16:         remove j* from S
17:     else if dt = dt_arrival then
18:         add arriving jobs to S
19:     end if
20: end while
```

Once the while loop ends, we can calculate the $j_{complete} - a_j$ for each job j to find the delay and take the average of all of these to calculate the mean delay.

3.1.1 Pitfall of discrete event simulation

Although the naive event-driven implementation is relatively simple to understand, it is inefficient because it advances time in very small increments, making it impractical to run at scale. In our above description, our simulator will have to advance time at every discrete point in our rank function, even if nothing interesting happens along the way. For example, if a job has 100 rank points between its current age and completion, the naive simulator must step through all 100, even if no other event (arrival or completion) occurs in between. This leads to unnecessary work.

3.2 Our design: bisection-driven discrete event simulation

A more efficient approach is to allow multiple jobs to advance farther in larger jumps. Instead of stepping through every break point one at a time, each job will be moved to the last relevant rank break-point before the next event occurs.

3.2.1 The Core Algorithm Overview

Rather than asking "when is the next event?", we ask a different question: Given the current set of jobs and their ages, which rank can we advance them to before any event occurs?

To answer this question, we make note of a key structural design where only a single job can be between any two consecutive rank breakpoints at a time. In other words, the identity of the running job does not change within that phase. This means breakpoints are the only candidate preemption points, and between any two consecutive breakpoints, the simulation is entirely predictable. So, they are the only moments we need to explicitly simulate.

Given a discrete rank function $r(a)$ defined by points $(a_1, r_1), (a_2, r_2), \ldots, (a_n, r_n)$. Define a rank phase as the interval of ages $[a_i, a_{i+1})$ over which a job's rank is constant at r_i. As the current job starts aging within this phase, there are two events that can happen:

1. There is an arrival while the job is being served.
2. The job completes before reaching age a_{i+1}.

If we can determine which two boundary points the next occurrence of any of these two event occurs within, we can skip directly to point with the smaller age.

In case 1, the next preemptible time is when the job's rank jumps to r_{i+1}. In case 2, the simulation would run all the current jobs to r_i, then allow the job to complete the rest of its service until it completes.

If neither an arrival, nor a completion occurs during the phase, the running job simply reaches age a_{i+1}, and its rank changes to r_{i+1}. This may change which job has the lowest rank, but this simulation does not treat this as a separate event to step to.

3.2.2 Group Advance

Since rank is a function of only age, "advancing jobs to rank r^*" becomes clear: each job j with current rank $r_{curr} < r^*$ and current age x_j simply needs to be aged forward to the unique age at which its rank function first reaches r^*. We call this a group advance to r^*. Let us also define S as the set of active jobs that have not completed yet.

After the group advance, every job has rank at least r^*, and the new lowest-rank job becomes the next candidate to run. The naive simulator would have processed each break point crossing along the way as its own event; however, our simulator can collapse all of those crossings into a single update. As a result, the only events the simulation needs to stop at explicitly are arrivals and completions. Our simulator is able to do this through a process called bisection.

Algorithm 3.2. Group Advance

```
for each job j ∈ S do
    target_age ← rank_inv(j.age, r*)
    if j.age + j.remaining ≤ target_age then
        j.age ← j.size                          ▷ job completes before reaching r*
    else
        j.age ← target_age
    end if
end for
```

Note: the group advance is computed inline. I.e, for each job, we invert the rank function to find the target age the next time it reaches rank r^*, capping at the job's size if it completes before reaching it.

3.2.3 Bisection

To find the next rank to advance the jobs to, we used bisection. Given budget T (time until next arrival), we want the largest rank r from the sorted rank value list $r_1, r_2, \ldots, r_n$ such that group advancing to r costs at most T and causes no completions. We binary search over this list:

Algorithm 3.3. Rank Bisection

```
lo, hi ← 0, len(rank_values) − 1
while lo < hi do
    mid ← ⌊(lo + hi + 1)/2⌋
    c ← cost(rank_values[mid])
    if c > T or any job completes reaching rank_values[mid] then
        hi ← mid − 1                            ▷ too far, search lower
    else
        lo ← mid                                ▷ reachable cleanly, try higher
    end if
end while
r* ← rank_values[lo]
group_advance(r*)
```

We binary search over the sorted list of all rank values $r_1, r_2, \ldots, r_n$. For each candidate rank r_{mid}, we evaluate $\text{cost}(r_{\text{mid}})$: If $\text{cost}(r_{\text{mid}}) > T$, the threshold is too high — we cannot reach it within budget, so we search lower. If $\text{cost}(r_{\text{mid}})$ would cause any job to complete, we also search lower, since a completion changes the job set and must be handled as an explicit event. Otherwise, r_{mid} is reachable cleanly, so we search higher. The result $r^* = r_values[lo]$ is the highest rank reachable within budget T with no completions and no change in job set.

Since the bisection finds the highest complete rank level reachable *within* budget, it may be the case that we have some leftover time $T_{rem} = T - \text{cost}(r^*)$, where $\text{cost}(r^*)$ is the time it takes to move every job to this point. By the definition of $group_advance(r^*)$, *all* jobs cannot reach the next rank threshold without going over the budget, but it may be possible for a partial subset of the lowest-rank jobs to make progress toward the next breakpoint. The simulator will handle the

remaining time with a small event-driven step by looping through jobs by a lowest-rank basis and advancing the job to the next breakpoint until either:

1. The leftover budget T_{rem} is exceeded.
2. A job is completed.

If a completion occurs, the simulator will record the completion time, and return to the simulation loop. If the job reaches the rank break point, then the scheduler is at a preemptable point and will recompute which job should run next. If the leftover budget is exhausted first, the simulator can simply advance to the next event. This leftover step is necessary because bisection only finds the largest full rank level that can be reached safely. It does not account for partial progress toward the next rank level. The local step handles this final fractional portion of the interval while preserving the same event semantics as the naive event-driven simulator.

3.2.4 Putting it all together

Since arrival times of the next job is the only events whose timing is fixed regardless of scheduling decisions, we structure our outer loop around inter-arrivals intervals. Each loop uses the time until the next arrival as its budget, and within each, the simulator resolves any completions and rank transitions.

Algorithm 3.4. Simulation

$S \leftarrow \emptyset, time \leftarrow 0$
while True **do**
    Simulate the next job, j
    $t \leftarrow$ next arrival time, S $\cup j$
    $T_budget \leftarrow$ next arrival time - time
    $r^* \leftarrow$ largest rank reachable by group advance within T_budget with no completion (found via bisection)
    $group_advance(S, r^*)$
    $T_rem \leftarrow T_budget - cost(r^*)$
    $t_used \leftarrow leftover_step(S, T_rem)$ ▷ returns time actually consumed
    $time \mathrel{+}= cost(r^*) + t_used$
end while

4 Results

To evaluate how the simulator scales with increasingly large discrete rank functions, we introduced a resolution parameter k. Given an original rank dataset $[(a_1, r_1), (a_2, r_2), \ldots, (a_n, r_n)]$, the parameter k increases the number of rank breakpoints by approximately a factor of k.

For each adjacent pair of ages in the original dataset, the transformation inserts $k-1$ intermediate age points by averaging neighboring ages. The transformed dataset contains $k(n-1)+1$ entries instead of n. The rank values themselves are duplicated across these intermediate points, preserving the overall shape of the rank function while increasing its discretization density.

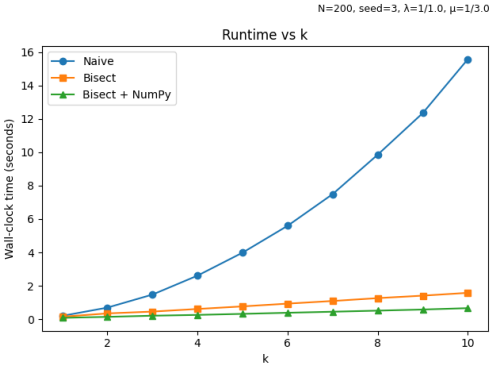
As k increases, the simulator must process a larger number of rank transitions and breakpoint crossings. This allows us to measure how the runtime of different simulation approaches scales as the rank function becomes more finely discretized.

For our results, we refer and compare the 3 algorithms below:

- **Naive:** The Naive simulator described in Section 3.1 follows a fully event-driven approach, explicitly going through every rank breakpoint, arrival, and completion event.
- **Bisect:** The Bisect simulator from Section 3.2 improves on this by using group advancement and binary search over rank levels to skip directly to the largest reachable rank threshold before the next event occurs.
- **Bisect + NumPy:** The Bisect+NumPy implementation preserves the same bisection-driven approach from Section 3.2 but replaces many per-job Python loops with vectorized NumPy array operations. In particular, rank lookups, breakpoint searches, and age updates are computed using ndarray operations and `np.searchsorted`, reducing the overhead of the Python interpreter.

4.1 Comparing Performance on Varying Resolution Discrete Rank Data

In this experiment, we evaluated the runtime performance of Naive, Bisect, and NumPy + Bisect simulators as the discretization parameter k varied from 1 to 10. To ensure a fair comparison, for each test, all algorithms were run using the same random seed, producing identical job arrival and size sequences across all trials. The underlying rank function was generated from a pre-provided dataset consisting of 400 age-rank points, with the parameter k controlling the effective resolution of the discretized rank function, and each algorithm had $N = 200$ jobs.



k	Naive (s)	Bisect (s)	Bisect+NP (s)
1	0.2038	0.1519	0.0860
2	0.6971	0.3493	0.1430
3	1.4770	0.4584	0.2120
4	2.6114	0.6150	0.2599
5	3.9873	0.7694	0.3222
6	5.5954	0.9353	0.3879
7	7.4925	1.0931	0.4506
8	9.8531	1.2645	0.5155
9	12.3423	1.4129	0.5822
10	15.5667	1.5808	0.6652

Figure 4.1. Naive vs Bisect vs NumPy+Bisect Runtime on pre-provided rank data for k=1 to k=10

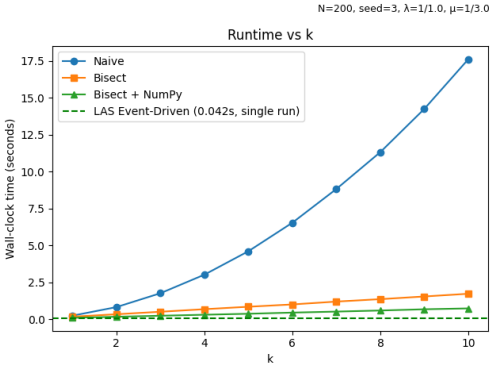
For the pre-provided rank dataset in Figure 4.1, the Naive simulator increased from about 0.2 seconds at $k = 1$ to over 15.57 seconds at $k = 10$. In contrast, both the Bisect and Bisect+NumPy had significantly faster results. Bisect increased from about 0.15 seconds at $k = 1$ to over 1.58 seconds at $k = 10$. Bisect+NumPy increased from about 0.09 seconds at $k = 1$ to over 0.67 seconds at $k = 10$. This demonstrates that the bisection-based approach is significantly faster when we increase the resolution of our rank data.

Overall, the results show that the Bisect and Bisect+NumPy simulator consistently outperform the Naive simulator and scale considerably better as the number of rank breakpoints increases. The gap between the three approaches widens with larger k, indicating that the optimization becomes more beneficial at higher discretization resolutions. There is a slight advantage of using Bisect+NumPy over Bisect for a small number of jobs as it uses vectorized NumPy operations instead of per-job python loops to reduce operations overhead.

4.2 Comparing Performance on Event driven LAS

To compare the algorithms' performance on a different rank function, we used Least Attained Service (LAS) rank function as described in Section 2. LAS serves as a natural performance baseline because it has a simple rank function. Its event driven implementation requires no rank lookup or bisection, making it the theoretical lower bound on simulation complexity.

We kept the same number of jobs ($N = 200$) and same random seed as Section 4.1 to produce identical job arrival and size sequences. Our rank data was produced by `rank data = [(i/10, i/10) for i in range(500)]` such that the rank increases as each job's age increases and varied k from $k = 1$ to $k = 10$. The results are shown in Figure 4.2 below.



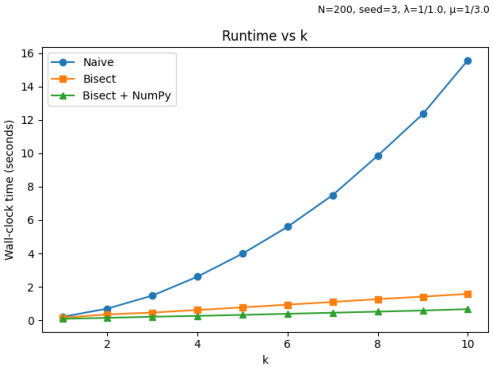
k	Naive (s)	Bisect (s)	Bisect+NP (s)
1	0.2429	0.1742	0.0986
2	0.8245	0.3402	0.1699
3	1.7640	0.5092	0.2388
4	3.0212	0.6828	0.3068
5	4.6097	0.8486	0.3738
6	6.5422	1.0050	0.4516
7	8.8267	1.1983	0.5198
8	11.3264	1.3632	0.5941
9	14.2553	1.5443	0.6771
10	17.6219	1.7288	0.7370

Figure 4.2. Naive vs Bisect vs Bisect + NumPy Runtime on LAS rank data for k=1 to k=10

In Figure 4.2 we see a similar trend as the pre-provided dataset where the Naive implementation is drastically slower than both Bisect and Bisect+NumPy. At $k = 10$, the Naive LAS simulator reaches approximately 17.62 seconds whereas Bisect reaches about 1.73 and Bisect + NumPy reaches only about 0.74 seconds. Again, we see a similar trend of Bisect+NumPy being slightly faster than Bisect.

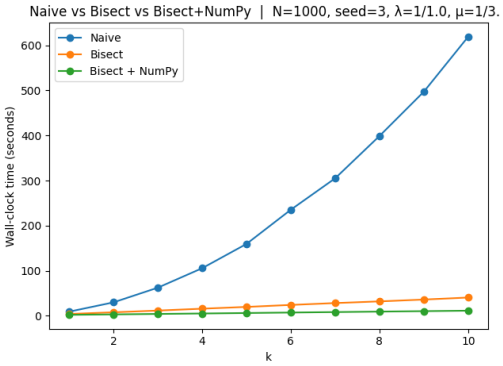
4.3 Comparing Performance on Varying Number of Jobs

To evaluate how each simulator scales with the number of jobs, we fixed the rank function to the pre-provided 400-point dataset and varied $N \in \{200, 1000, 5000\}$ while sweeping the resolution parameter k from 1 to 10. As before, all three algorithms were run using the same random seed and identical job sequences for each trial.



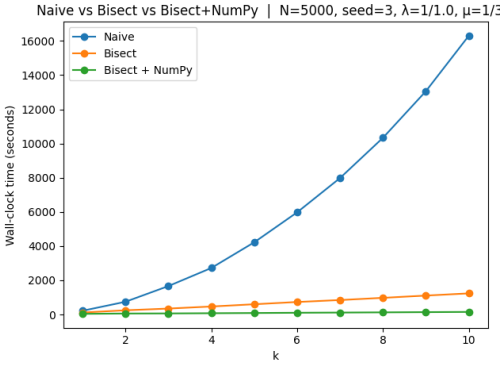
k	Naive (s)	Bisect (s)	Bisect+NP (s)
1	0.2038	0.1519	0.0860
2	0.6971	0.3493	0.1430
3	1.4770	0.4584	0.2120
4	2.6114	0.6150	0.2599
5	3.9873	0.7694	0.3222
6	5.5954	0.9353	0.3879
7	7.4925	1.0931	0.4506
8	9.8531	1.2645	0.5155
9	12.3423	1.4129	0.5822
10	15.5667	1.5808	0.6652

Figure 4.3. Runtime comparison of Naive, Bisect, and Bisect+NumPy simulators for varying rank resolution with $N = 200$ jobs.



k	Naive (s)	Bisect (s)	Bisect+NP (s)
1	9.1193	3.9745	2.0699
2	29.7374	7.6872	3.0443
3	62.3187	11.5447	4.0730
4	105.5081	15.6967	5.0462
5	159.3676	19.6532	6.0946
6	235.2695	24.0629	7.1637
7	304.9969	28.1962	8.2009
8	398.9087	31.9125	9.2674
9	497.1473	35.9855	10.2479
10	619.1202	40.4549	11.3149

Figure 4.4. Runtime comparison of Naive, Bisect, and Bisect+NumPy simulators for varying rank resolution with $N = 1000$ jobs



k	Naive (s)	Bisect (s)	Bisect+NP (s)
1	218.5638	115.2240	38.4978
2	743.3325	245.9618	57.0586
3	1661.0148	346.6715	62.3019
4	2718.3989	467.2685	74.1766
5	4220.2312	599.0800	86.5476
6	5973.9350	731.5631	101.4913
7	7981.0204	845.4156	113.1008
8	10334.0320	974.7109	125.2073
9	13060.2977	1106.3379	138.4455
10	16303.6642	1235.3012	150.1999

Figure 4.5. Runtime comparison of Naive, Bisect, and Bisect+NumPy simulators for varying rank resolution with $N = 5000$ jobs

The results across Figures 4.3, 4.4, and 4.5 have the same pattern: as N grows, the Naive simulator slows down dramatically, reaching approximately 15.57 seconds at $N = 200$, 619.1 seconds at $N = 1000$, and over 16,303 seconds at $N = 5000$ when $k = 10$. The Bisect simulator scales substantially better, ending at roughly 1.58, 40.5, and 1,235 seconds respectively. Most notably, Bisect+NumPy achieves the best performance across all job counts, reaching only 0.67, 11.3, and 150.2 seconds at $k = 10$ for $N = 200$, 1000, and 5000 respectively. This shows that Bisect and Bisect+NumPy perform significantly better than Naive when there is both more discretized resolution and more jobs.

Bisect and Bisect + NumPy have slight runtime variation when there are less jobs ($N = 200$) but we can start to see a clear difference when we increase the number of jobs to $N = 1000$ and $N = 5000$. Bisect + NumPy is significantly faster by ending at 150.2 seconds for $N = 5000$ jobs compared to Bisect which takes 1235.3 seconds.

5 Conclusion

In this project, we developed and analyzed several simulators for job queues governed by discrete rank functions. Starting from a straightforward event-driven implementation, we identified that explicitly processing every event results in substantial computational overhead as the size of our rank data and jobs increases. To address this issue, we designed a bisection-driven simulator that advances groups of jobs directly to reachable rank thresholds, avoiding many unnecessary intermediate events.

Our results show that the bisection-based approach scales significantly better than the naive simulator as both the rank resolution parameter k and the number of jobs N increase. While the naive simulator experiences dramatic runtime growth, the Bisect and Bisect+NumPy implementations remain substantially more efficient. Further, the NumPy-vectorized implementation of Bisect achieves the best performance for a large number of jobs and a larger rank dataset.

Overall, these results demonstrate that combining group advancement, bisection, and vectorized computation using NumPy provides an optimized algorithm for efficiently simulating scheduling systems with large sizes of rank data and jobs.

Acknowledgements

We would like to thank Professor Ziv Scully and Amit Harlev for their patience in teaching us, leading insightful weekly discussions, and giving us valuable feedback which helped our project get where it needed to be. This research would not be possible without their continued support and guidance.

References

[1] Ziv Scully, *A New Toolbox for Scheduling Theory*, Carnegie Mellon University, 2022.